

Project Development – Wetterstation

HTL-Wels | ITP

Brückl Matthias, Aistleitner Daniel, Bergbauer Klaus

Project close down: 10.06.2026

1. Original Task

The project was initiated as an ITP school assignment. The original idea came from Matthias Brückl's mother, who wanted to monitor the exact temperature in her garden. From this starting point, the team developed a full outdoor weather station with the following original objectives:

- Read out multiple sensors: UV, temperature, air pressure, humidity, and wind speed (anemometer)
- Web-based data analysis and visualization
- Energy-independent power supply (solar)
- Minimalist 3D-printed case

The planned hardware at project start included a Raspberry Pi Zero 2W, BME280, DHT22, VEML6075, breadboard, and PETG filament. The tech stack was planned as React (frontend), Python (backend), SQLite (database), and a web server.

2. What Has Changed Over the Course of the Project?

Several significant changes occurred from the original plan to the final product:

Hardware changes:

- The **VEML6075** UV sensor was replaced by the **VEML7700**
- The **BME280** air pressure sensor was dropped entirely from the final build
- The **solar power supply** was removed and replaced with a **battery/DC concept** — the energy independence goal was fundamentally redesigned

- A **self-built anemometer** using the **US5881LUA** Hall effect sensor was added, which was not in the original plan at all
- The team started with a **borrowed Raspberry Pi Zero 2W** and only received their own unit mid-project

Software changes:

- A **hotspot mode** was added so the Raspberry Pi can create its own WLAN access point when no network is available — this was not in the original scope
- A **WLAN management UI** was built directly into the web interface (settings page)
- The frontend evolved from a basic React layout into a full **dashboard with radial charts, detail views, and dark mode**

Project management changes:

- The milestone for "Finished Database and APIs" was revised from 26.03.2026 to 03.06.2026 — a significant delay
- "Finished Frontend" was similarly pushed from 29.04.2026 to 03.06.2026
- The WBS was updated at version 14 to reflect the new anemometer work packages
- The project remained according to plan throughout in terms of team organisation and communication, with no major crises reported

3. How Has Project Management Changed Over the Course of the Project?

At the start, project management was primarily focused on setting up structure - defining work packages, filling the handbook, and establishing communication rules (weekly Monday meetings, GitHub as the only file sharing platform).

As the project progressed, the management style shifted from planning to adaptive coordination. When hardware arrived late, when the solar concept was dropped, and when milestones had to be revised, the team responded pragmatically rather than sticking rigidly to the original plan. The weekly status reports became the core coordination tool, with each team member taking turns writing them.

The revised WBS (version 14, March 2026) reflects the most significant management change: the formal recognition that the anemometer was a new, substantial work stream that needed its own work packages, not just an adjustment to existing ones.

4. Individual Reflections

Matthias Brückl (Project Manager, Backend Developer)

What have I learned?

Managing a technical project as both project manager and the main backend developer was challenging. I learned that these two roles pull in different directions — as a developer you want to go deep into a problem, but as a project manager you need to keep the overview. I also learned a lot technically: setting up systemd services, building a FastAPI backend, and connecting everything end-to-end on the Raspberry Pi.

What worked well?

The decision to use FastAPI with uvicorn turned out to be the right call. It made the API development significantly cleaner and the integration with the React frontend much more straightforward. The weekly status reports also worked well — they forced reflection and kept the whole team informed even when not everyone was working on the same thing at the same time.

What would I do differently in the next project?

I would separate the project manager and developer roles more clearly from the beginning, or at least protect dedicated time for management tasks. In this project, the backend work sometimes made it hard to keep the project handbook up to date or to plan ahead. I would also define the power supply concept earlier — the solar-to-battery switch cost time and caused a ripple effect on the case design and the milestone plan.

Anything else?

The project turned out significantly better than I expected when we started. Building a working, deployable weather station with a real web interface, persistent data storage, and auto-start on boot is something I'm genuinely proud of.

Daniel Aistleitner (Frontend Developer)

What have I learned?

I deepened my React and TypeScript skills considerably over the course of this project. Before this, I had built websites but nothing with live data from a real API. Learning how to connect the frontend to a backend, handle loading states, and visualize sensor data with Recharts was very valuable. I also learned more about SQLite and logging from my early work on the database and the logging script.

What worked well?

The incremental approach to the frontend worked well. Starting with a basic framework, then adding non-functional features, then hooking up the real data meant that at every stage we had something working and demo-able. The dashboard design came together nicely once we settled on the radial chart approach.

What would I do differently in the next project?

I would define the API contract between frontend and backend earlier. In this project, the frontend was sometimes developed against placeholder data, and when the real API came in, adjustments were needed. A simple agreed-upon API

spec at the start would have saved time. I would also start the WLAN management feature earlier - it was added late and felt rushed.

Anything else?

Working with real hardware added a dimension that pure software projects don't have. Seeing sensor values actually appear on a webpage you built yourself is a different feeling than displaying static mock data.

Klaus Bergbauer (Hardware, 3D Modelling)

What have I learned?

I learned a huge amount about electronics, sensor wiring, and the physical side of embedded systems. Planning and building a self-designed anemometer from scratch using a Hall effect sensor was something I had never done before. I also learned Tinkercad for 3D modelling and had to iterate multiple times on the case design to get the dimensions right and to accommodate the anemometer.

What worked well?

The decision to build a custom anemometer rather than buy one turned out well — it was cheaper, it fit the case better, and it gave me a much deeper understanding of how the sensor data is actually generated. The test-print approach (print a small section first, check dimensions, then print the full part) saved a lot of filament and time.

What would I do differently in the next project?

I would measure more carefully before modelling. Several test prints had to be redone because of small dimensional errors that could have been caught earlier with more precise measurements. I would also involve the software team earlier in the case design process — there were moments where a decision I made about the case layout affected where cables could run, which created extra work for the wiring.

Anything else?

The physical, tangible result of this project makes it stand out compared to purely software-based school projects. Having an actual device that you can hold, mount outside, and watch collect real weather data makes all the effort feel worth it.